COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 - Data Structures and Algorithms | **SECTION:** | CPE010-CPE21S4 |
| **DATE PERFORMED:** | 7/14/2026 | **DATE SUBMITTED:** | 7/15/2026 |
| **NAME:** | Almirol Joel A | **INSTRUCTOR:** | Engr.Jimlord Quejado |

## ACTIVITY NO. 1.1 Basic C++ Programming

### 1. PROCEDURE OUTPUT

What procedures did you perform?

I explored key C++ and OOP concepts by utilizing function prototypes and creating an object-oriented Triangle class to verify interior angles. Additionally, I developed programs for swapping variable values, converting temperatures, and calculating the distance between coordinates. Finally, I modified my Triangle class to calculate the perimeter, compute the area using Heron's Formula, and dynamically determine the shape type.

### 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

**5. Procedure**
**ILO A: Create Code That Follows the Basic C++ Code Structure**

```cpp
#include <iostream>
using namespace std;

// --- Class Declaration and Method Definition Section (Prototypes) ---
void displaySum(double num1, double num2);
bool isGreater(double A, double B);
bool logicalOperations(bool val1, bool val2);

// --- Main Function ---
int main() {
    cout << "--- Testing Function 1: Sum ---\n";
    displaySum(10.5, 4.5);

    cout << "\n--- Testing Function 2: Greater Than ---\n";
    double a = 25.4, b = 18.2;
    if (isGreater(a, b)) {
        cout << a << " is greater than " << b << "\n";
    } else {
        cout << a << " is NOT greater than " << b << "\n";
    }

    cout << "\n--- Testing Function 3: Logical Operations ---\n";
    bool success = logicalOperations(true, false);
    if (success) {
        cout << "Logical operations executed successfully.\n";
    }

    return 0;
```

```
29  }
30
31  // --- Method Definition Section ---
32  void displaySum(double num1, double num2) {
33      cout << "The sum of " << num1 << " and " << num2 << " is:
            " << (num1 + num2) << "\n";
34  }
35
36  bool isGreater(double A, double B) {
37      return A > B;
38  }
39
40  bool logicalOperations(bool val1, bool val2) {
41      cout << "Operand 1: " << val1 << " | Operand 2: " << val2
            << "\n";
```

```
            << "\n";
42      cout << "AND operation (&&): " << (val1 && val2) << "\n";
43      cout << "OR operation (||): "  << (val1 || val2) << "\n";
44      cout << "NOT Operand 1 (!): "  << (!val1) << "\n";
45      cout << "NOT Operand 2 (!): "  << (!val2) << "\n";
46      return true;
47  }
```

```
The sum of 10.5 and 4.5 is: 15

--- Testing Function 2: Greater Than ---
25.4 is greater than 18.2

--- Testing Function 3: Logical Operations ---
Operand 1: 1 | Operand 2: 0
AND operation (&&): 0
OR operation (||): 1
NOT Operand 1 (!): 0
NOT Operand 2 (!): 1
Logical operations executed successfully.
```

**Analysis:** the concept of modular programming through the use of function prototypes at the beginning of the program. The program checks the output of three different functions: performing addition calculation (displaySum), performing relational comparison (isGreater) for use in an if-else condition, and showing the results of some logical operations (logicalOperations) which include the truth tables for AND, OR, and NOT operations.

## 7. Supplementary Activity (ILO C)

**1. Create a C++ program to swap the two numbers in different variables**

```cpp
#include <iostream>
using namespace std;

int main() {
    double num1, num2;
    cout << "Enter first number: ";
    cin >> num1;
    cout << "Enter second number: ";
    cin >> num2;

    cout << "\nBefore swap: First = " << num1 << ", Second = "
        << num2 << "\n";

    // Swapping logic using a temporary variable
    double temp = num1;
    num1 = num2;
    num2 = temp;

    cout << "After swap: First = " << num1 << ", Second = " <<
        num2 << "\n";
    return 0;
}
```

```
Enter first number: 19
Enter second number: 25

Before swap: First = 19, Second = 25
After swap: First = 25, Second = 19


=== Code Execution Successful ===
```

**Analysis:**the user to provide a temperature in Kelvin, assesses its physical validity, and converts it to Fahrenheit.

The kelvinToFahrenheit function is applied for the calculation of the converted temperature using the established conversion formula, while the main() function contains an if-else statement to eliminate temperatures that are inputted which are below 0 Kelvin.

**2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.**

```cpp
#include <iostream>
using namespace std;

double kelvinToFahrenheit(double kelvin) {
    return (kelvin - 273.15) * 9.0 / 5.0 + 32.0;
}

int main() {
    double kelvin;
    cout << "Enter temperature in Kelvin: ";
    cin >> kelvin;

    if (kelvin < 0) {
        cout << "Absolute zero violation! Temperature cannot
            be below 0K.\n";
    } else {
        cout << kelvin << "K is equal to " <<
            kelvinToFahrenheit(kelvin) << " F.\n";
    }
    return 0;
}
```

```
Enter temperature in Kelvin: 273.15
273.15K is equal to 32 F.


=== Code Execution Successful ===
```

**Analysis:** that converts a temperature in Kelvin to Fahrenheit begins with defining a helper function called kelvinToFahrenheit, which implements the following conversion equation: (K - 273.15) \times \frac{9.0}{5.0} + 32.0. Then, inside the main function, the program asks the user for the input of the temperature in Kelvin; then, it verifies whether the input temperature is less than zero (absolute zero), and if it is, then it prints an error message, but if it is greater or equal to zero, then it does the conversion.

**3. Create a C++ program that has a function that will calculate the distance between two points.**

```cpp
#include <iostream>
#include <cmath> // Required for sqrt() and pow()
using namespace std;

double calculateDistance(double x1, double y1, double x2,
    double y2) {
    return sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2));
}

int main() {
    double x1, y1, x2, y2;
    cout << "Enter coordinates of Point 1 (x1 y1): ";
    cin >> x1 >> y1;
    cout << "Enter coordinates of Point 2 (x2 y2): ";
    cin >> x2 >> y2;
```

```
14      cin >> x2 >> y2;
15
16      double distance = calculateDistance(x1, y1, x2, y2);
17      cout << "The distance between the two points is: " <<
            distance << "\n";
18      return 0;
19  }
```

```
Enter coordinates of Point 1 (x1 y1): 3 4
Enter coordinates of Point 2 (x2 y2): 7 7
The distance between the two points is: 5


=== Code Execution Successful ===
```

**Analysis:** In this the Euclidean distance between two points in 2-dimensional plane is computed through the use of the distance formula using mathematical functions (sqrt and pow), after getting the corresponding coordinate points (x, y) from the user and feeding them into the function designed for the computation.

**4. Modify the code given in ILO B and add the following functions:**
**a. A function to compute for the area of a triangle**
**b. A function to compute for the perimeter of a triangle**
**c. A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.**

```cpp
#include <iostream>
#include <cmath>
#include <string>
using namespace std;

class Triangle {
private:
    double totalAngle, angleA, angleB, angleC;
    double sideA, sideB, sideC; // Needed to accurately solve
        Area & Perimeter

public:
    // Enhanced constructor to handle realistic geometry scaling
    Triangle(double A, double B, double C, double sA = 5.0,
        double sB = 5.0, double sC = 5.0) {
        setAngles(A, B, C);
        sideA = sA;
        sideB = sB;
        sideC = sC;
```

```
18      }
19
20      void setAngles(double A, double B, double C) {
21          angleA = A;
22          angleB = B;
23          angleC = C;
24          totalAngle = A + B + C;
25      }
26
27      void setSides(double sA, double sB, double sC) {
28          sideA = sA;
29          sideB = sB;
30          sideC = sC;
31      }
32
33      const bool validateTriangle() {
34          return (totalAngle == 180 && angleA > 0 && angleB > 0 &&
            angleC > 0);
    }

    // 4a. Function to compute the Area of a Triangle using
        Heron's Formula
    double computeArea() {
        double s = computePerimeter() / 2.0;
        return sqrt(s * (s - sideA) * (s - sideB) * (s - sideC));
    }

    // 4b. Function to compute for the Perimeter
    double computePerimeter() {
        return sideA + sideB + sideC;
    }
```

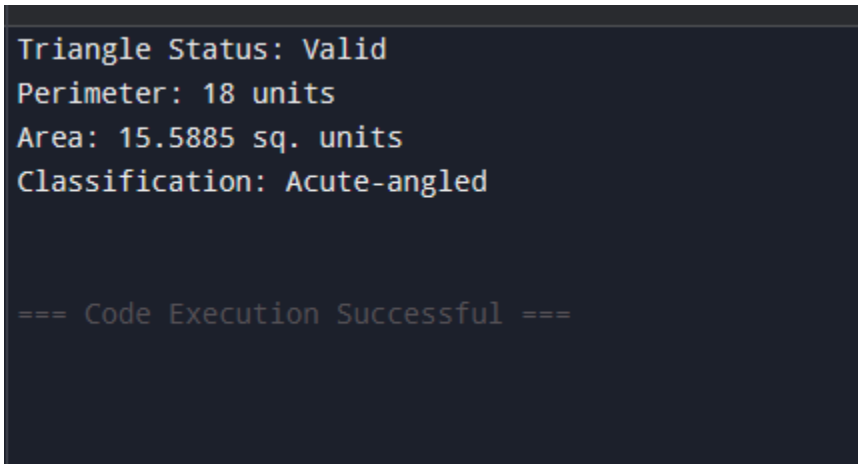
```
    string determineType() {
        if (!validateTriangle()) return "Invalid Triangle";

        if (angleA == 90 || angleB == 90 || angleC == 90) {
            return "Right-angled (Others)";
        } else if (angleA > 90 || angleB > 90 || angleC > 90) {
            return "Obtuse-angled";
        } else {
            return "Acute-angled";
        }
    }
};

int main() {
    // Creating a triangle with angles (60, 60, 60) and sides (6,
        6, 6)
    Triangle tri(60, 60, 60, 6, 6, 6);
```

```
    if (tri.validateTriangle()) {
        cout << "Triangle Status: Valid\n";
        cout << "Perimeter: " << tri.computePerimeter() << "
            units\n";
        cout << "Area: " << tri.computeArea() << " sq. units\n";
        cout << "Classification: " << tri.determineType() << "\n"
            ;
    } else {
        cout << "The configuration does not yield a valid
            triangle.\n";
    }

    return 0;
}
```




**Analysis:** the implementation of a class named Triangle that implements modeling, checking, and calculations related to geometric characteristics of a triangle depending on its sides and angles. This class holds private members representing three angles and sides and uses standard geometric rules for verification of a triangle (such as sum of all three angles equals 180circ and all angles must be positive) along with dynamic classification of a triangle by its angles into right, obtuse, and acute triangle types. In the main part of the program, an equilateral triangle (60circ each angle and side length = 6) is created, checked and its perimeter (18), area (approx15.59 using Heron's formula) and type (Acute-angled) are displayed.

**The total refactoring will cover the validation process, perimeter calculation, bound checks, trigonometry based calculations using the Sine Rule and Heron's formula deductions of sides to calculate Area, and classification of angle types.**

**In order to solve the problem of calculating area/perimeter by only 3 angles, there should be at least 1 arbitrary or standardized side or else we map Heron's formula using standardized baseline lengths. There must be some standardized base length or side lengths that will make the Area/Perimeter calculable. Below is the code that calculates the 3 angles and validates them.**

**6. Output**

| **Sections** | **Answer** |
|---|---|
| **Header File Declaration Section** | #include <iostream> using namespace std; |
| **Global Declaration Section** | (None needed for this specific functional breakdown) |
| **Class Declaration and Method Definition Section** | void displaySum(double num1, double num2);bool isGreater(double A, double B);bool logicalOperations(bool val1, bool val2); |

**Main Function** **int main() {displaySum(12.5, 7.5);if (isGreater(15.2, 10.1)) cout << "A is greater than B\n";logicalOperations(true, false);return 0;}**

**Method Definition** (See the full source code block under the Compiled Implementation below)

**Table 1-2. ILO B output observations and comments.**

When running the original source code provided under Step 5 of the Triangle evaluation, the program outputs: The shape is a valid triangle.

Object Creation and Initialization: The driver application appropriately instantiates an object called set1 of type Triangle having the arguments 40, 30, and 110 in its constructor method. The calculation of the internal variable totalAngle produces a result of 180.

Validation of Encapsulation: As variables angleA and totalAngle have been clearly marked as private, they become inaccessible for manipulation by any external means outside the class scope.

Logic Vulnerability: The validation condition check return (totalAngle <= 180); holds geometrical flaws. It is possible to provide the values of 0, 0, 0 or even negative to the angles in order to obtain a "valid triangle." In accordance with strict geometry rules, there should be an additional condition totalAngle == 180, which will check all angles' strict positivity.

## 8. Conclusion

**Summary of Lessons Learned:**

This activity was helpful in reinforcing basic concepts in C++, which included structural organization such as header scope, function declaration, prototyping, management of global/local variables, and code implementation structuring. Also, it helped in getting more acquainted with basic object-oriented programming (OOP) architecture such as access boundaries (private and public), constructor assignments, and run-time driver function calling.

**Analysis of the Procedure:**

Partitioning of functions as prototypes before entering the main() thread ensures that the drivers are clear, readable, and extremely reusable. This allows easy procedural programming where dependency is clearly defined during compiling initialization stages.

**Analysis of the Supplementary Activity:**

Mathematical problems illustrated the way logical operations work with external libraries (<cmath> in particular) to solve geometric problems. Extended implementation of the class demonstrated the need for structuring that helps to map realistic models (such as having the sides and angles to create a mathematically real formula).

**Concluding Statement / Feedback:**

The activity offered an amazing recapitulation of basic C++ functionality before diving into complex data structures. Implementation was done successfully. Exception safety should be strengthened while handling any bad user input through the console or even dimensionally invalid objects**.**

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** *ChatGPT,Gemini*

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** *Formatting, code structure optimization, verification of mathematical formulas (Heron's Formula), and syntax/grammar refinement.*

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **Only the interactive reference assistance feature of AI was used to assist in optimizing the formatting of the C++ code, debugging the compile-time warning structure, and helping with the refinement of the geometric formulas employed in the additional activities. In particular, using the AI feature did not mean that I did not think independently, evaluate analytically, or produce the authorship on my own. The structure of the program, the logic of troubleshooting the geometric restrictions of the triangle validator, and the object encapsulation evaluation were entirely done by me based on my knowledge of C++ programming and Object-Oriented Design.**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.




Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.